HANDS-ON WORKSHOP · ABOUT 2 HOURS

Linux, Networks & Git

The everyday developer toolkit, from your first command to your first push.

$ ls $ dig $ git push

[Presenter name] · [Date]

AGENDA

Three parts, about two hours

PART 1 · 50 MIN

Linux & the CLI

Filesystem & core commands	25
Lab 1: command-line drills	7
Permissions, links, processes	10
Env vars & shell scripts	8

PART 2 · 30 MIN

Networking

Layers, IP, subnets, DHCP, DNS, TCP/UDP	15
The web: HTTP, REST, TLS	10
Lab 2: poke the network	5

PART 3 · 27 MIN

SSH & Git

SSH & keys	5
Git: commits, branches, remotes	15
Lab 3: your first push	7

+ welcome 5 · break 5 · wrap-up 3 = 120 min

90-minute cut: labs become live demos, skip sed/awk and HTTP versions, no break.

BEFORE WE START

What everyone needs on screen

A Linux shell

Ubuntu in a VM, WSL2 on Windows, or a free cloud VM. A macOS terminal works for most of Parts 1 and 2.

```
PS> wsl --install
dash.freestyle.sh (free VM)
```

A GitHub account

Needed in Part 3, when you push your first repo over SSH. The free plan is fine.

witr: optional, recommended

“Why is this running?” Not required, but it helps a lot, today and in the next session.

apt on Ubuntu 26.04+, else the .deb from GitHub releases

```
$ sudo apt update
$ sudo apt install -y git curl dnsutils traceroute htop tree jq tldr
$ git --version # if this prints a version, you’re ready
```

01

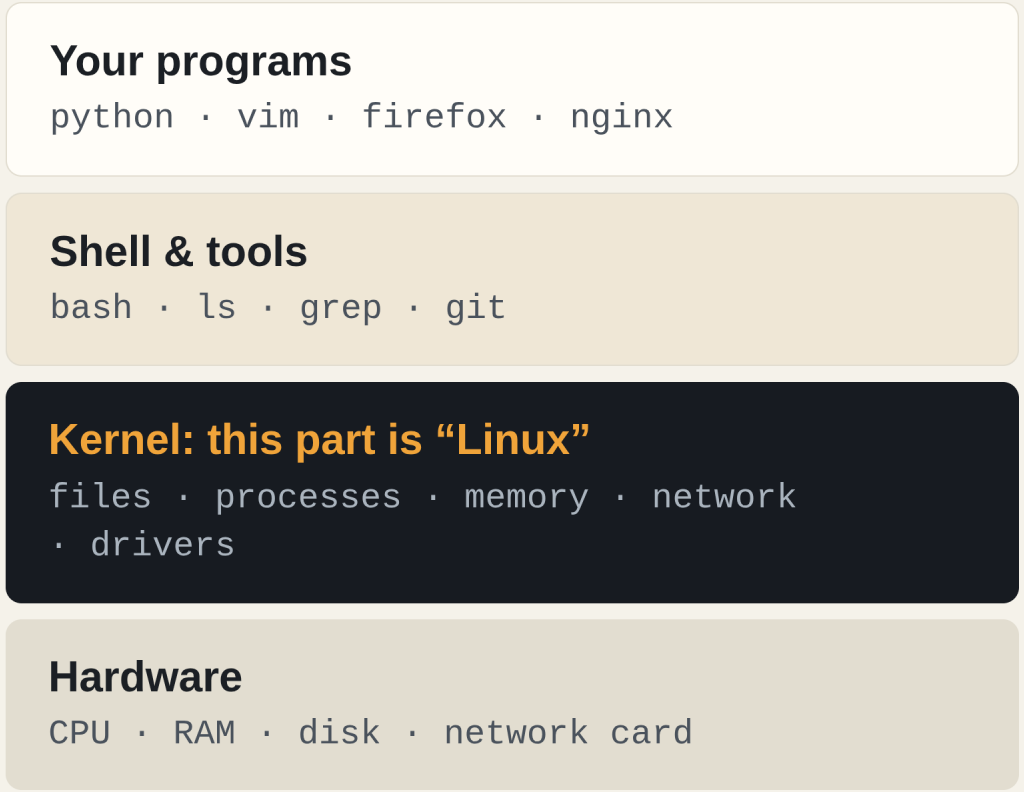
PART 1 · 50 MINUTES

Linux & the command line

filesystem · commands · pipes · permissions · processes · scripts

LINUX · BASICS

What is Linux?



A kernel, not a whole OS

Linux manages the hardware and shares it fairly between programs. Linus Torvalds started it in 1991.

Distros bundle it for you

Ubuntu, Debian, Fedora, Arch = kernel + tools + a package manager (apt, dnf, pacman).

It runs most of the world

Most servers and cloud VMs, Android phones, Docker containers, and all of the top 500 supercomputers.

LINUX · DIRECTORY STRUCTURE

One tree, starting at /

/

The root. No C: or D: drives; disks are mounted somewhere inside the tree.

/home/you

Your files. ~ is a shortcut for it.

/etc

System configuration, as plain text files you can read.

/var

Data that changes. Logs live in **/var/log**.

/tmp

Scratch space for anyone. Usually wiped on reboot.

/dev

Devices as files: disks (sda), terminals, and **/dev/null**, the black hole.

/usr/bin

Installed programs: ls, grep, git and friends.

/proc

Live kernel and process info, generated on the fly.

Try: `ls / · tree -L 1 / · cat /etc/os-release · cat /proc/cpuinfo`

LINUX · THE CLI

Anatomy of a command

```
sara@laptop:~$ ls -la /etc
```

prompt: user @ machine : folder

command

options (flags)

argument

Terminal

The window that shows text and sends your keystrokes.

Shell

The program inside it that reads and runs commands: bash, zsh.

Why bother?

Faster, scriptable, repeatable, and it works over SSH on any server.

LINUX · HANDY COMMANDS 1/2

Moving around & managing files

`pwd`	Where am I?
`ls -la`	List everything, hidden files and details too
`cd dir · cd .. · cd -`	Go in · go up · go back to the last folder
`mkdir -p a/b/c`	Make folders, parents included
`touch notes.txt`	Create an empty file
`cp -r src dst`	Copy (-r for folders)
`mv old new`	Move or rename
`rm -r dir`	**Delete. There is no trash can.**

Paths: absolute ones start at / · relative ones start where you are · **.** here · **..** parent · **~** home

```
$ mkdir -p ~/workshop/notes
$ cd ~/workshop
$ pwd
/home/sara/workshop
$ touch notes/day1.txt
$ cp notes/day1.txt notes/day2.txt
$ mv notes/day2.txt notes/old.txt
$ ls notes
day1.txt old.txt
$ rm notes/old.txt
```

LINUX · HANDY COMMANDS 2/2

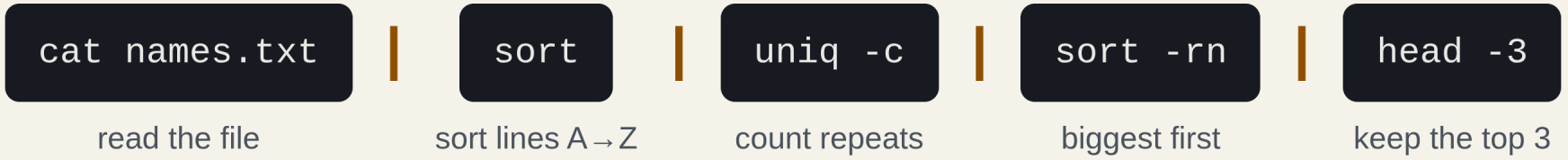
Reading & searching

`cat file`	Print the whole file
`less file`	Scroll through it: / to search, q to quit
`head -n 5 · tail -n 20`	First 5 lines · last 20 lines
`tail -f app.log`	Follow a log live (Ctrl+C to stop)
`wc -l file`	Count lines
`grep "error" app.log`	Show lines that match
`grep -rn "TODO" .`	Search every file below here, with line numbers
`grep -i · -v · -c`	Ignore case · non-matching lines · count
`find . -name "*.log"`	Find files by name

```
$ wc -l /etc/passwd
38 /etc/passwd
$ grep "bash" /etc/passwd
root:x:0:0:root:/root:/bin/bash
sara:x:1000:1000::/home/sara:/bin/bash
$ find ~ -name "*.txt"
/home/sara/workshop/notes/day1.txt
$ sudo tail -f /var/log/syslog
# Ctrl+C to stop following
```

LINUX · PIPES & REDIRECTION

Small tools, chained with |



Redirect output into files

`cmd > out.txt`	Write output (overwrites!)
`cmd >> out.txt`	Append to the end
`cmd 2> err.txt`	Write only the errors
`cmd > /dev/null 2>&1`	Silence everything

```
$ ls /etc | wc -l
# how many entries in /etc?
$ ls /etc > etc.txt
$ grep conf etc.txt | head -5
$ history | grep ssh
# “what was that ssh command?”
```

LINUX · TEXT TOOLS

sed & awk: three one-liners each

sed: find & replace

```
sed 's/http:/https:/g' urls.txt
```

Replace every match, print the result

```
sed -i 's/8080/9090/' app.conf
```

-i edits the file in place (careful)

```
sed -n '10,20p' big.log
```

Print only lines 10 to 20

awk: work with columns

```
awk '{print $1}' access.log
```

First column (split on spaces)

```
awk -F: '{print $1}' /etc/passwd
```

Split on : and list every username

```
awk '$9 == 404' access.log
```

Only rows where column 9 is 404

```
$ history | awk '{print $2}' | sort | uniq -c | sort -rn | head -5
# your 5 most-used commands: everything from today in one line
```

LINUX · BASH HISTORY & SHORTCUTS

Type less, find more

Tab

Complete commands and paths. Twice: show options.

↑ / ↓

Previous / next command.

Ctrl + R

Search your history as you type.

Ctrl + C

Stop the running command. Not copy!

Ctrl + L

Clear the screen.

Ctrl + A / E

Jump to the start / end of the line.

history

Numbered list of past commands.

!! · !42

Rerun the last command · command #42.

```
$ apt install htop # Permission denied?
$ sudo !! # = sudo apt install htop
```

LINUX · HELP & PACKAGES

Getting help, installing software

Ask the system

```
$ man ls # the manual, q quits
$ ls --help # short summary
$ tldr tar # examples, not essays
$ type cd # builtin, alias or file?
$ which python3 # where is it?
```

Install with apt

```
$ sudo apt update # refresh the list
$ apt search htop
$ sudo apt install htop
$ sudo apt remove htop
$ apt list --installed | grep git
```

⚠ **sudo** runs one command as root, the administrator. Read the whole command before you press Enter.

LAB 1 · 7 MINUTES

Command-line drills

01 Create ~/workshop/logs and ~/workshop/notes with one command.
`hint: mkdir -p, {a,b}`

02 Which distro and version are you running?
`hint: /etc/os-release`

03 How many user accounts exist on this machine?
`hint: /etc/passwd, wc`

04 List every .conf file directly inside /etc.
`hint: find -maxdepth 1 -name`

05 Save the list of /etc into ~/workshop/notes/etc.txt.
`hint: >`

06 From that file, show only the names containing “net”.
`hint: grep`

07 Find your 5 most-used commands.
`hint: history | awk | sort | uniq`

08 Bonus: the 5 biggest items in /var/log.
`hint: du -ah, sort -rh`

LINUX · FILE PERMISSIONS

Who can read, write, run

```
-   rwx  r-x  r--
```

type: - file, d folder, l link	user (owner)	group	others

`r = 4 · w = 2 · x = 1`

`7 = rwx · 5 = r-x · 4 = r--`

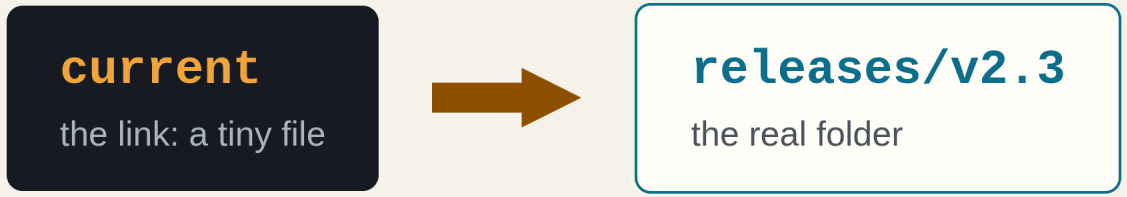
`chmod 754`

or symbolic: chmod u+x, g-w, o-r

```
$ ls -l deploy.sh
-rw-r--r-- 1 sara devs 512 Oct 17 18:30 deploy.sh
$ chmod +x deploy.sh # now it can run: ./deploy.sh
$ chmod 600 secrets.env # rw- --- --- : only me
$ sudo chown sara:devs report.txt # new owner : group
```

LINUX · SYMBOLIC LINKS

Symlinks: a pointer to another path



Why use them

Switch versions instantly; keep one stable path for many files.

They can break

Delete the target and the link points at nothing.

```
$ ln -s releases/v2.3 current
$ ls -l current
lrwxrwxrwx 1 sara sara 13 current -> releases/v2.3

$ ln -sfn releases/v2.4 current # repoint
$ readlink current
releases/v2.4
```

Bonus (recommended): inodes A filename just points to an inode, which holds the data and metadata. Hard link = two names, one inode. Try ls -i, stat file, df -i.

LINUX · PROCESS MANAGEMENT

What’s running, and why?

`ps aux`	Snapshot of every process
`ps aux | grep nginx`	Find one by name
`top · htop`	Live view of CPU and memory, q quits
`pgrep -a python`	Process IDs (PIDs) by name
`kill 1234`	Politely ask PID 1234 to stop
`kill -9 1234`	Force it: last resort

witr: “why is this running?”

Explains where a process came from: its parent chain, the service that started it, and the ports it holds.

```
$ witr nginx
$ witr --port 8080
$ witr --pid 1234 --tree
# the classic way:
$ sudo ss -tlnp | grep 8080
```

LINUX · ENVIRONMENT VARIABLES

Settings every program inherits

```
$ echo $HOME
/home/sara
$ echo $PATH
/usr/local/bin:/usr/bin:/bin
$ NAME=sara # shell-only variable
$ bash -c 'echo $NAME' # a child process
# (empty line: the child can’t see it)
$ export NAME
$ bash -c 'echo $NAME'
sara
$ export PORT=8080 # set + export
```

export

A plain variable lives only in this shell. export copies it into the environment, so every program you start from here inherits it.

$PATH

Folders searched for commands, left to right. “command not found” = not in PATH.

~/.bashrc

Runs in every new shell: keep exports and aliases here. Reload with source ~/.bashrc.

DATABASE_URL

Apps read config and secrets from env vars instead of hard-coding them.

LINUX · SHELL SCRIPTS

A script is commands saved in a file

backup.sh

```
#!/usr/bin/env bash
set -euo pipefail # stop at the first error

SRC="${1:-$HOME/workshop}"
DEST="$HOME/backups"
STAMP=$(date +%F_%H%M)

mkdir -p "$DEST"
tar -czf "$DEST/$STAMP.tar.gz" "$SRC"
echo "Backup saved: $STAMP"
```

`#!/usr/bin/env bash`

The shebang: which program runs this file

`$1 · ${1:-default}`

First argument, with a fallback value

`$(date +%F)`

Insert a command’s output

```
$ chmod +x backup.sh
$ ./backup.sh ~/workshop/notes
Backup saved: 2026-10-17_1830
```

02

PART 2 · 30 MINUTES

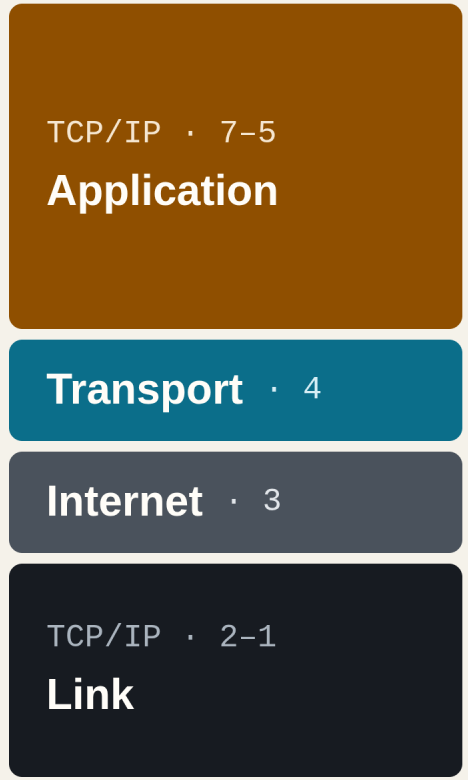
Networking, from the shell

layers · ip & mac · subnets · dhcp · dns · tcp/udp · http · rest · tls

NETWORK · OSI MODEL

The 7 OSI layers, top to bottom

7	**Application**	what your app speaks	`HTTP, DNS, SSH`
6	**Presentation**	encoding, compression, encryption	`TLS, JSON, JPEG`
5	**Session**	open and keep a conversation	`sessions, RPC`
4	**Transport**	ports, reliability, ordering	`TCP, UDP`
3	**Network**	IP addresses and routing	`IP, ICMP, routers`
2	**Data link**	delivery on the local network	`Ethernet, MAC`
1	**Physical**	bits as signals	`cables, Wi-Fi radio`



NETWORK · LAYERS

OSI on paper, TCP/IP in practice

TCP/IP layer	OSI layer	What lives here	Address	Try it
Application	7 · 6 · 5	HTTP, DNS, SSH	URL, hostname	curl, dig
Transport	4	TCP, UDP	port	ss -tuln
Internet	3	IP, ICMP (ping)	IP address	ip addr, ping
Link	2 · 1	Ethernet, Wi-Fi, ARP	MAC address	ip link, ip neigh

Each layer wraps the one above it, like envelopes:



NETWORK · ADDRESSES

MAC, IP, port: three kinds of address

MAC address

`3c:22:fb:7a:10:9e`

Built into the network card. Only used inside your local network.

Like your face: known in this room.

IP address

`192.168.1.42`

Where you are on the network. Changes when you join a new Wi-Fi.

Like a street address.

Port

`22 · 53 · 80 · 443`

Which program on that machine: 0 to 65535. SSH, DNS, HTTP, HTTPS.

Like an apartment number.

IPv4: 32-bit, nearly used up · **IPv6**: 128-bit (2001:db8::42), about half of traffic · **IPv8**: a 2026 draft proposal, not a standard

```
$ ip -br addr # brief: interface, state, IP
wlan0 UP 192.168.1.42/24
$ ip -br link # the MAC address
wlan0 UP 3c:22:fb:7a:10:9e
```

NETWORK · SUBNETS & CIDR

Reading 192.168.1.42/24

192 . 168 . 1 . 42 /24

Network part: first 24 bits, the same for the whole LAN

Host part: last 8 bits, 256 addresses

CIDR	Mask	Addresses	Typical use
/32	255.255.255.255	1	one single host
/24	255.255.255.0	256	home or office LAN
/16	255.255.0.0	65,536	campus, cloud VPC
/8	255.0.0.0	16.7 million	10.0.0.0/8 range

Private ranges

`10.0.0.0/8`
`172.16.0.0/12`
`192.168.0.0/16`

Same subnet: talk directly. Different subnet: go through the gateway (router).

NETWORK · DHCP

How your laptop gets an IP: DORA

D

Discover

laptop → everyone

“Is there a DHCP server here?”

O

Offer

router → laptop

“You can have 192.168.1.42.”

R

Request

laptop → router

“I’ll take that one, please.”

A

Acknowledge

router → laptop

“It’s yours for 24 hours.”

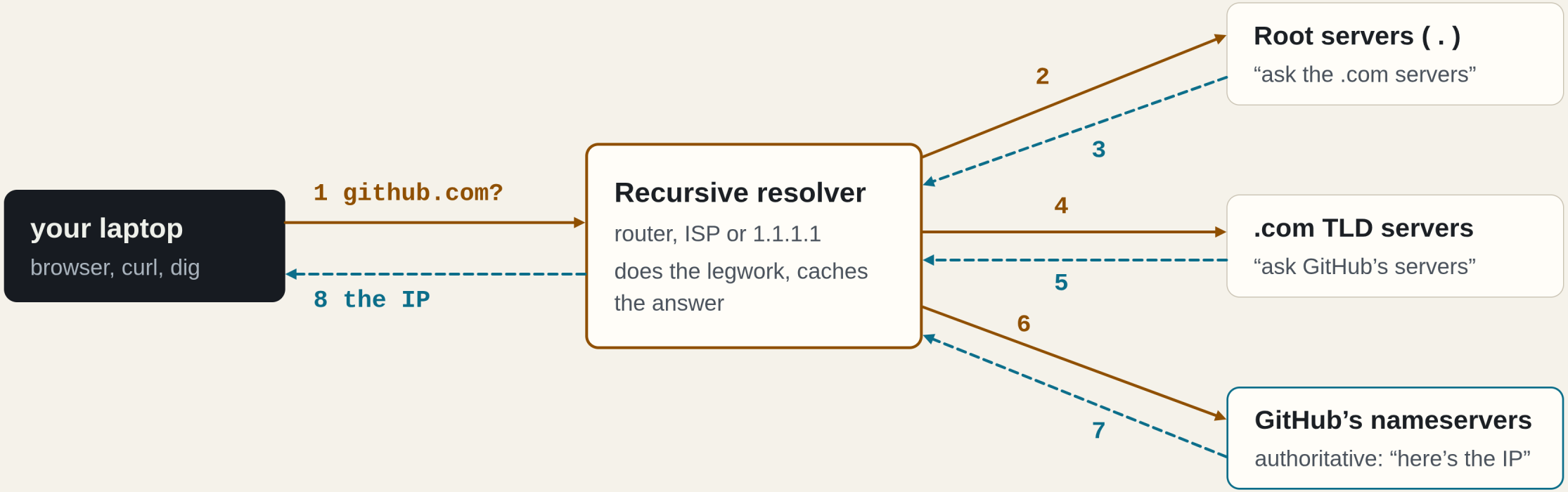
What you get back

- IP address + subnet mask (/24)
- Default gateway: your router
- DNS servers to ask for names
- Lease time: when to renew

```
$ ip route | grep default
default via 192.168.1.1 dev wlan0 proto dhcp
$ resolvectl status | grep "DNS Server"
Current DNS Server: 192.168.1.1
```

NETWORK · DNS

How a name is resolved, step by step



Solid = question, dashed = answer. Your laptop asks once (a recursive query); the resolver asks root → TLD → authoritative, then caches the result for its TTL. Watch it live: dig +trace github.com

NETWORK · DNS

DNS records, and how to query them

Record	Means	Example value
A	name → IPv4	203.0.113.10
AAAA	name → IPv6	2001:db8::10
CNAME	alias → another name	myapp.com
MX	where email goes	mail.myapp.com
NS	who is authoritative	ns1.myapp.com
TXT	text: verification, SPF	"v=spf1 …"

```
$ dig +short github.com
# → one or more IPv4 addresses
$ dig github.com AAAA
$ dig github.com NS +short
$ dig -x 1.1.1.1 +short
one.one.one.one.
$ dig @1.1.1.1 github.com # ask a
specific resolver
$ cat /etc/hosts # checked first
```

NETWORK · TRANSPORT

TCP vs UDP: a phone call vs a postcard

	TCP	UDP
Setup	3-way handshake first	none: just send
Delivery	guaranteed and in order; lost data is resent	best effort: may drop or reorder
Cost	more overhead and latency	lightweight and fast
Used by	web (HTTP/1.1 and 2), SSH, email, Git	DNS, video calls, games, HTTP/3 (QUIC)

The TCP handshake

`client → SYN`

“Can we talk?”

`SYN-ACK ← server`

“Yes. Can you hear me?”

`client → ACK`

“Yes.” Connected.

Try: `ss -tuln` (who’s listening?) · `nc -zv github.com 443` (is this port open?)

NETWORK · HTTP

HTTP is text: a request, then a response

```
$ curl -v http://example.com
# what curl sends:
> GET / HTTP/1.1
> Host: example.com
> User-Agent: curl/8.5.0
> Accept: */*
# what comes back:
< HTTP/1.1 200 OK
< Content-Type: text/html
<!doctype html> …
```

Method + path

GET reads, POST creates, PUT/PATCH update, DELETE removes.

Headers

Metadata: Host, Content-Type, Authorization, Cookie…

Status codes

2xx	success: 200 OK, 201 Created
3xx	go elsewhere: 301 Moved, 304 Not Modified
4xx	your mistake: 400, 401, 403, 404
5xx	server’s mistake: 500, 502, 503

NETWORK · HTTP VERSIONS

HTTP/1.1 → 2 → 3: same meaning, faster delivery

	HTTP/1.1 · 1997	HTTP/2 · 2015	HTTP/3 · 2022
Runs on	TCP	TCP	QUIC, over UDP
Format	plain text	binary frames	binary frames
Requests	one at a time per connection	many streams on one connection	many independent streams
Weak spot	slow when a page has many files	one lost packet stalls every stream	some networks block UDP

```
$ curl -sI https://www.google.com | head -1
HTTP/2 200
```

NETWORK · REST

REST APIs: resources + HTTP verbs

GET	/users	list users	200
GET	/users/42	one user	200
POST	/users	create a user	201
PUT	/users/42	replace it	200
PATCH	/users/42	change some fields	200
DELETE	/users/42	delete it	204

Nouns in the URL, verbs in the method. Stateless, JSON in and out, status codes carry the result.

```
$ API=https://jsonplaceholder.typicode.com
$ curl -s $API/users/1 | jq '.name, .email'
"Leanne Graham"
"Sincere@april.biz"

$ curl -s -X POST $API/posts \
  -H "Content-Type: application/json" \
  -d '{"title":"hello"}'
{ "title": "hello", "id": 101 }
```

NETWORK · TLS

TLS: the S in HTTPS

Encryption

Nobody on the café Wi-Fi can read it.

Integrity

Nobody can change it on the way.

Identity

A certificate, signed by a trusted CA, proves it's really github.com.

The handshake, simplified (TLS 1.3)

1. **Client hello:** the ciphers I support, plus my key share.
2. **Server hello:** my key share and my certificate.
3. **Both** derive the same secret key; normal HTTP flows, encrypted.

```
$ curl -vI https://github.com 2>&1 \
  | grep -iE "tls|subject|issuer"
# TLS version, who the cert is for, who signed it
$ openssl s_client -connect github.com:443 -brief
# Ctrl+C to quit
```

LAB 2 · 5 MINUTES

Poke the network

01 Find your IP, subnet and default gateway.
`hint: ip -br addr, ip route`

02 ping 1.1.1.1, then ping github.com. What extra step happened?
`hint: names need…`

03 Resolve github.com, then reverse-lookup 1.1.1.1.
`hint: dig +short, dig -x`

04 How many hops (routers) to 1.1.1.1?
`hint: traceroute`

05 Start a web server, then fetch it from a second terminal tab.
`hint: python3 -m http.server 8000`

06 Which process owns port 8000?
`hint: ss -tlnp, witr --port`

07 Which HTTP version does github.com answer with?
`hint: curl -sI … | head -1`

08 Bonus: GET user 1, then POST a new post to jsonplaceholder.
`hint: the REST slide`

03

PART 3 · 27 MINUTES

SSH & Git

keys · remote shells · commits · branches · remotes · push

SSH · SECURE SHELL

SSH: a terminal on another machine

Private key

`~/.ssh/id_ed25519`

Stays on your laptop. Never share it, never commit it.

Public key

`~/.ssh/id_ed25519.pub`

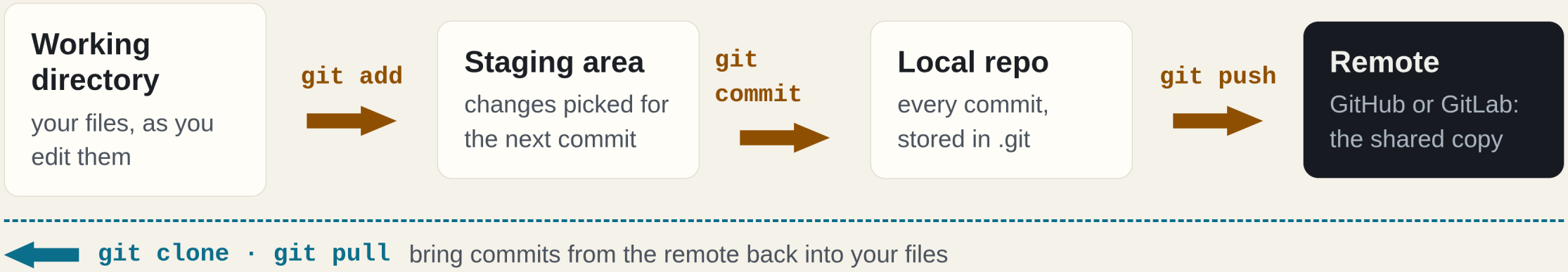
A lock you hand out to servers and GitHub. Only your private key opens it.

```
~/.ssh/config
Host lab
  HostName 203.0.113.10
  User sara
→ now just: ssh lab
```

```
$ ssh-keygen -t ed25519 -C "sara@laptop" # creates the key pair in ~/.ssh
$ ssh-copy-id sara@203.0.113.10 # puts your public key on the server
$ ssh sara@203.0.113.10 # log in, no password needed now
$ scp report.pdf sara@203.0.113.10:~/ # copy a file over SSH
```

GIT · THE MENTAL MODEL

What is Git? Four places your code lives



Git = snapshots with history

Each commit is a snapshot of the whole project. Go back, compare, or work in parallel on branches.

Git is not GitHub

Git is the tool on your machine. GitHub hosts repos and adds pull requests, issues and CI.

GIT · SETUP

One-time setup, then a repo

Tell Git who you are (once per machine)

```
$ git config --global user.name "Your Name"
$ git config --global user.email "you@example.com"
$ git config --global init.defaultBranch main
```

Give GitHub your SSH key

```
$ cat ~/.ssh/id_ed25519.pub
# GitHub › Settings › SSH and GPG keys
$ ssh -T git@github.com
Hi sara! You've successfully authenticated…
```

Start or copy a repository

```
$ git init # new repo in this folder
$ git clone git@github.com:user/repo.git
$ git status
On branch main
```

GIT · DAILY COMMANDS

The everyday loop: status, add, commit

```
$ git status
Changes not staged for commit:
  modified: backup.sh
$ git diff # what exactly changed?
$ git add backup.sh # or: git add .
$ git commit -m "Add timestamp to backups"
[main 3f2a1c9] Add timestamp to backups
$ git log --oneline
3f2a1c9 Add timestamp to backups
a81b02e Initial commit
```

Commit small, commit often

One logical change per commit: easy to review, easy to undo.

Messages say why

“Fix login redirect loop”, not “stuff”.

.gitignore

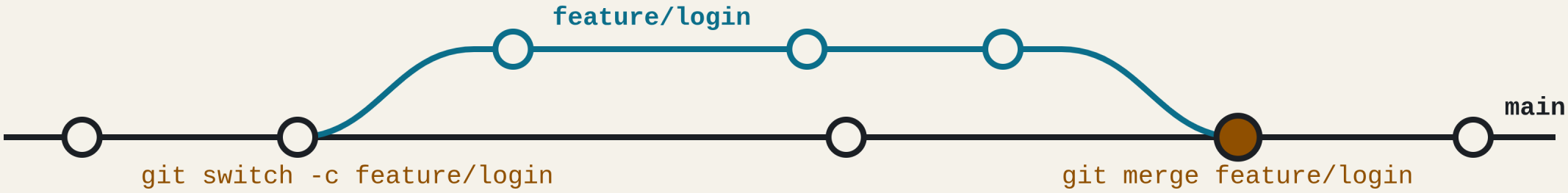
Keep out secrets and junk: .env, node_modules/, build/, *.log

Undo, safely

git restore file drops edits; git restore --staged file unstages.

GIT · BRANCHES

Branches: work without breaking main




```
$ git branch # list; * = current
$ git switch -c feature/login
# …edit, add, commit…
$ git switch main
$ git merge feature/login
$ git branch -d feature/login
```

Merge conflict? Git marks the spot

```
<<<<<<< HEAD
port = 8080
=======
port = 9090
>>>>>>> feature/login
```

Keep what you want, delete the markers, then add and commit.

GIT · REMOTES

Remotes: share your work

```
$ git remote add origin git@github.com:sara/workshop.git
$ git remote -v
origin git@github.com:sara/workshop.git (fetch)
origin git@github.com:sara/workshop.git (push)
$ git push -u origin main # first push links the branch
$ git push # every time after that
$ git pull # get teammates' commits
$ git fetch # download only, merge later
```

origin is just a nickname for the remote's URL. **Pull before you push.**

The team flow

1. Pull the latest main
2. Create a branch
3. Commit, then push the branch
4. Open a pull request
5. Review, then merge into main

LAB 3 · 7 MINUTES

Your first push

01 Create an empty GitHub repo called workshop (no README).
`hint: github.com/new`

02 In ~/workshop, start a repo and commit backup.sh.
`hint: git init, add, commit -m`

03 Ignore your backup files (*.tar.gz) with a .gitignore.
`hint: echo … > .gitignore`

04 Check your SSH key works, add the remote, push main.
`hint: ssh -T, git remote add, push -u`

05 On a new branch, add README.md, commit, push the branch.
`hint: git switch -c, git push -u`

06 Open a pull request, merge it, then update your local main.
`hint: git switch main, git pull`

07 Look at the history, as a graph.
`hint: git log --oneline --graph`

08 Bonus: cause a merge conflict on purpose, then fix it.
`hint: edit one line on GitHub and locally`

WRAP-UP

You made it. Keep going.

You can now

- Move around and search files from the shell
- Chain small tools with pipes
- Read and change permissions
- Find what’s running, and why
- Debug a network from the terminal
- Push code to GitHub over SSH

Keep practising

> **Faradars: Git, GitHub & GitLab course**
> Strongly recommended for really understanding Git

OverTheWire: Bandit

Learn Linux by playing a wargame over SSH

Learn Git Branching

Interactive, visual Git exercises

The Missing Semester (MIT)

Shell, scripting, Git and tools, in depth

explainshell.com

Paste any command, see what each part does